

Encoding the Jersey Covid-19 Gathering Control Order in L4

Audience: Matthew Waddington (primary), and anyone who shared his concerns about encoding legislation and subordinate instruments as executable rules.

Subject matter: *Covid-19 (Gathering Control) (Jersey) Order 2020* (R&O.166/2020 as amended; last in force before repeal), encoded in L4 next to parallel files for the first enacted text and a mid-chain consolidation.

Companion sources (this workspace):

File	Scope
GCO-first-version.l4	First enacted text (flat 10-person cap)
GCO-as-at-20210115.l4	Mid-chain consolidation (indoor/outdoor split; pre-last amendment tail)
GCO-as-repealed.l4	As last in force before repeal (indoor 10 / outdoor 20; Art.5 wedding/funeral caps)

Official page (historical): https://www.jerseylaw.je/laws/current/ro_166_2020

Status of this report: frank after-action note, not a prospectus. Where the stack helps, we say so. Where the problem is hard because *any* formalisation of law is hard, we say so too.

1. Why this Order, and what the challenge was asking for

The Gathering Control Order (GCO) is a small, self-contained subordinate instrument with:

- a short interpretation / ontology (gatherings, premises, classes of person);
- numerical thresholds that *changed under amendment*;
- defeasible exceptions (school groups; weddings / civil partnerships / funerals; work);
- pure prohibitions (`must not take part` / `must not allow`);
- criminal offences with a fixed maximum fine;
- two named *force majeure*-style residual defences that deliberately ship open texture out to external guidance (critical workers) or a very concrete factual class (Visit Jersey industrial tour escorts).

It is therefore a good stress sample for everything Matthew has long pressed for from "rules as code":

1. **Isomorphism** — can a lawyer audit the encoding against the statute clause by clause, rather than reverse-engineering a black-box spreadsheet?
2. **Amendment honesty** — when the 10-person cap became 20 outdoors and stayed 10 indoors, does the formalisation show the *diff*, not just a rewritten monolith?
3. **Open texture** — do we pretend critical-worker status is computeable from first principles, or do we leave a typed hole that evaluation must fill?
4. **Constitutive vs regulative weight** — "must not" here is mostly *conduct that is an offence if...*, not a multi-stage commercial duty with successive HENCE branches. Does the tool force the wrong model, or allow the light one?
5. **Drafter authority** — can people who write Orders of this kind *own* the model eventually, or only receive it as a gift from programmers?

6. **Explainability** — when a case is evaluated, can you see *why* Art.3 fired?

What this Order does *not* ask us to do is also informative: there is little temporal process (no "within 14 days deliver..."), almost no multi-party choreography, and no secondary legislation tables deep enough to need a full decision-table notation. So the GCO is a **level-1** (ontology + boolean decisions + numeric thresholds) exercise, not a **level-3** contract-process exercise. That is a feature for pedagogy: we can show progress without drowning the reader in HENCE/LEST.

2. Encoding process (what we actually did)

The work followed statute-order, not engineer-order.

Step 1 — Read the Order as a structure, not a bag of rules

From the text we extracted three layers:

1. **Entities and facts** (Art.1 / interpretation habits): gathering size; indoor vs outdoor; school-group flag; wedding/funeral ceremony roles and guest counts; public accessibility of premises; who is "responsible"; the two residual status flags.
2. **Constitutive regimes** (Arts.3–7): when a gathering *is* restricted; when a responsible person is under a must-not-allow duty.
3. **Offence and penalty** (Art.9): taking part / allowing when the duty bites, subject to the named defences; level-2 fine (= £1,000 under the standard scale then in force for this encoding).

Finding: almost everything interesting in the machine encoding is already implicit in how a careful drafter *namespaces* nouns. The L4 `DECLARE` block is not a software invention; it is the Interpretation article made tabular.

Step 2 — Fix the umbrella types first

We declared:

- `Gathering` — Art.3 world (headcount + indoor/outdoor + exception flags
 - ceremony officer/guest split for Art.5).
- `PubliclyAccessiblePremisesGathering` — Art.7 world (indoor headcount on public premises + responsibility + allow/support fact).
- `Participant` / `ResponsiblePerson` — carry the residual force-majeure leaves.
- Situation records that pair a gathering with an actor for the offence questions.

Finding: keeping Art.3 and Art.7 as *different* records (rather than one giant "Event" type with many optional fields) mirrors the Order's own split. Optional soup is how encodings become unreadable.

Computed fields remain pure siblings of the facts: outdoors means not indoors; venue kind folds that boolean; officer+guest totals for Art.5 are a single derived number. That is all the "programming" most clauses need.

Step 3 — Encode exceptions as named boolean gates, not buried `OR` s

Art.4–6 became three named predicates:

- school-group exception applies

- wedding/funeral exception applies (availability **and** size cap respected)
- work exception applies

Art.3 restrain then becomes:

```
restricted ⇔ size exceeded ∧ ¬ (Art.4 v Art.5 v Art.6)
must not take part ⇔ restricted
```

Finding: naming the exception gates is the single highest return-for-effort improvement for audit. Buried nested ORs in a single mega-rule look "clever" and fail Matthew's review test the first time a reader tries to line-match the Order.

Step 4 — Keep residual open texture as typed holes

Critical-worker force majeure and the Visit Jersey industrial tour defence are boolean inputs, with @desc prose quoting (or paraphrasing) the Order's own reference to external guidance / factual class. Evaluation never *guesses* them.

Finding: this is the honest answer to "how do you encode open texture?". You don't. You type the residual and force every caller — human form, test case, or API — to assert it. Pretending NLP will resolve critical-worker status from the Order alone is where RaC projects lose lawyers.

Step 5 — Offence = duty breached ∧ act/omission ∧ ¬defence

Art.3 offence: restricted gathering, person not under the critical-worker force majeure leaf. Art.7 offence: must-not-allow duty, person allowed it, not under the tour-guide leaf. Art.9 fine is a trivial level-2 constant once an offence bit is true.

Finding: criminal "commits if" is a *constitutive* pattern on top of a prohibitive fact pattern. We did **not** force a full PARTY ... MUST ... WITHIN ... HENCE ... LEST ... process model. Light encodings should stay light; L4 allows both weights (see §5.2).

Step 6 — Parallel files for amendment history

Rather than one "always current" file with comments narrating history, we kept **three isomorphic skeletons** (first enacted, mid-chain, last-in-force) with -- DIFF: markers where numeric roofs and Art.5 structure moved. Tests include the cases that *switch outcome* under amendment (e.g. outdoor gathering of 11: prohibited first-enacted; lawful under the last outdoor-20 rule).

Finding: git history alone is not a legislation story. Drafters review by comparing *versions of the regime*, not commits. Parallel files plus DIFF markers are the lowest ceremony way to honour that without building a consolidator.

Step 7 — Golden cases before cleverness

Each file ends with concrete gatherings and paired #ASSERT / #EVAL style checks:

- indoor 11 ordinary → restricted / offence liable (absent defence);
- outdoor 11 ordinary → outcome *depends on version*;
- outdoor 21 ordinary → restricted under last-in-force;
- school group over size → exception saves;
- wedding respecting officer+guest roof → exception saves; exceeding roof does not;

- critical-worker force majeure leaf → no Art.3 offence even if restricted;
- Art.7 public premises indoor 11 allowed by responsible person → offence unless tour-guide leaf.

Finding: the tests are the *specificity of the encoding's claim*. Without them, "we encoded Art.3" is marketing.

Step 8 — Export the questions a non-author would ask

Five `@export` entry points (roughly):

- does the person commit an Art.3 offence?
- does the person commit an Art.7 offence?
- maximum fine given offence bit?
- must a person not take part under Art.3?
- must a responsible person not allow under Art.7?

Each `GIVEN` parameter has `@desc` suitable for an OpenAPI / MCP consumer with zero project context.

Finding: publishing is different from encoding. An encoding nobody can call from a form or an agent remains a research artefact.

3. Interesting findings along the way

3.1 The Order's english already nearly *is* a decision procedure

Read Art.3 with Arts.4–6: size exceedance, then carve-outs. There is almost no hidden control flow. Once thresholds and exception predicates are named, the L4 reads as a retyped Order, not an algorithm invented beside it. That validates the hypothesis that **good legislative structure is half the formalisation**.

3.2 The costly part is ontology, not connectives

"Indoor more than 10" is trivial boolean arithmetic. Getting *wedding pot officers vs guests vs total* right so that Art.5's later roof ($\text{officers} + \text{guests} \leq 20$ both indoors and out in the last text) attaches to the right numbers is where time went. Same story as every RaC pilot: **types and facts beat logic**.

3.3 Indoor/outdoor started life as one boolean and earned a kind

We kept `takes place indoors` as the authoritative input, derived `outdoors` and `VenueKind` from it. That avoids inconsistent states ($\text{indoors} \wedge \text{outdoors}$) and still names both poles the way the Order does after amendment. Dual independent booleans would have been a silent bug farm.

3.4 Parallel versions beat "one file with comments"

The outdoor-11 case is the existence proof. First-enacted and last-in-force *disagree*. Keeping both as living files makes the disagreement executable rather than narrative. A consolidating engine can come later; isomorphism and tests come first.

3.5 Regulative full power was intentionally under-used

L4 can express stateful obligations with event traces. The GCO's "must not take part" is better as a boolean duty-fact. Ramming it into a process model would have been tool-worship. Matching *weight of formal machinery to weight of the source* is itself a modelling skill — and a place Matthew's critique of over-

engineered RaC tools remains fair when ignored.

3.6 Mixfix / prose identifiers buy audit, not magic

the gathering is a restricted gathering under Article 3 is longer than isRestricted. The length is the feature: a legal reader diffs against Art.3 without a legend. That is L4-surface design, not "we have VS Code".

3.7 Residual defences expose the myth of "full automation"

The critical-worker defence cites guidance published by the Minister. The Visit Jersey tour-guide defence is almost a joke-length factual class. Both belong in the model as *booleans*. Neither is deduced from the rest of the Order by pure logic. Any claim that an encoding "decides GCO compliance end-to-end without human residual input" is false for this Order — and should be said out loud.

3.8 Visualiser-level propositional form is a strict *projection* of this model

Strip the record fields to already-filled booleans and the offence tree collapses purely into the L4 Visualiser / VizExpr propositional fragment Matthew asked for as an interchange IR. The full encoding is the same logic with *arguments* restored: the "(g)" after each predicate. That ladder is the pedagogy (§7).

4. Scorecard against the recurring concerns

Three columns, kept honest:

Concern	Mostly any formal language + IDE	Distinctively L4 / legal CNL design	Still open
Machine-check that the encoding is well-formed	Yes (parse, typecheck, run asserts)	Prose-shaped errors help a little	—
Diffable history of the encoding under amendment	Yes (git + parallel files)	§ skeleton matching statute numbering	Automatic consolidation from gazette events
Search / rename of a defined notion everywhere	Yes (LSP go-to-def, rename)	Backtick multi-word names match legislative labels	—
Golden cases re-run on every change	Yes	—	Choosing <i>legally</i> interesting golden cases is craft
Side-by-side statute PDF and rule	Yes (split editor)	—	Automatic pin of clause → § still manual
Audit "line X of encoding = Art.N of Order"	Partial (comments, names)	Yes — §, mixfix names, operator vocabulary	Automatic line-level backlinks
Ontology = interpretation section	Partial (any records)	Yes — DECLARE / MEANS reading order, computed defined terms	Teaching drafters to write it (§7)
Open texture as residual	Possible in any language	Yes — typed params + @desc as schema	Capturing real guidance corpora
Decide vs simulate over time	N/A for GCO	Full deontic process available when source needs it	Choosing the right weight every time
Explain a yes/no answer	Partial (#EVALTRACE)	Ladder / VizExpr propositional projection	Automatic NL judgements a Court would accept
Deployable form / agent API	Any web wrapper	Yes — @export → OpenAPI/MCP in the same file	Auth, operational governance
Non-programmer <i>authors</i> the rule	No (IDE alone is not enough)	CNL surface reduces friction	The hard open problem (§6)
Portable mon language across tools	No single tool attribution	VizExpr / "L4-lite" propositional IR aimed at this	Community adoption
The law is complete "on the chip" without residual judgement	No	No	Unsolvable in the general open-texture case

Bottom line of the scorecard

- **About half** of the *engineering* difficulties dissolve once you accept *any* formal language with a real editor, type checker, tests, and version control.
- **About a quarter** are improved specifically by L4's legal surface (statute isomorphism, ontology syntax, deontic option when wanted, export annotations).
- **About a quarter** remain open because they are not tooling problems: open texture residual to non-formal sources; *authorship* by people who rightly refuse to become programmers; institutional process around amendment consolidations; and the general truth that mapping English law into crystalline logic is a modelling craft, not a button.

5. What "any language + IDE" already solves

These should not be sold as L4 miracles. They are the baseline of software tooling applied carefully:

1. **You cannot silently drop a bracket.** The editor tells you.
2. **You cannot rename only half the uses of "restricted gathering".**
3. **Amendment attempts that break golden cases fail in CI or on `l4 run`.**
4. **Two people can work on different articles without overwriting each other** without merging Word documents by hand.
5. **A third party can evaluate the same inputs and get the same answer.** Reproducibility is not the same as correctness, but it is a prerequisite for criticising correctness productively.

If a pilot claims novelty for this list alone, treat the claim sceptically. Every serious formal-methods shop has owned this baseline for decades. RaC's value is not "we discovered version control".

6. What L4 (or a peer legal CNL) still buys you

These are *not* free with "Python in VS Code" or "Prolog with a bare editor":

1. **Statute-isomorphic surface.** Section markers, multi-word identifiers, legal operators, exception-shaped boolean definitions. A reviewer can walk Art.3 in the Order and the file without a translation table.
2. **Interpretation-section as types.** Product types, sum types, computed fields — the same intellectual objects as defined terms and exclusive enumerations, voiced for legal writers rather than PL academics.
3. **Two regulatory weights, one language.** Light constitutive "must-not / if" (GCO) *and* process standards with deadlines/traces when the source reaches for them (leases, SAFEs, delivery clauses). The GCO encoding deliberately stayed light; the heavier machinery was not deleted from the language to make the light path work— both exist.
4. **Clear export contract as part of the source.** `@export` / `@desc` is mundane engineering, but bolting it to prose-readable definitions means the same file is both the auditable artefact and the service contract. That bridging is where many RaC tools force a second disconnected layer.
5. **A propositional intermediate aimed at portability.** The L4 Visualiser / VizExpr layer is intentionally *not* "full L4 syntax". It is close to the L4-lite propositional IR Matthew has asked for so that other legal DSLs can target shared visual and evaluation tools. That is product architecture, not language fandom.

None of this makes *programming easy for non-programmers*. It makes formalisation *less insulting* to legal readers, which is a different and smaller claim.

7. What remains open (and why "easy for non-programmers" is the hard nut)

7.1 The NP-complete problem, said carefully

"Making programming easy for people who are not programmers" is not literally NP-complete in the Cook–Levin sense. It is, however, a problem whose difficulty has the same *shape* as intractable problems that people casually call NP-complete when they mean "we will not find a general cheap solution":

- the target users are domain *experts* — they *should* resist false simplification;
- every layer of sugar that hides structure also hides mistakes;
- every layer of exposure of structure reintroduces "this looks like code";
- human language and judgement occupy residual cells no type system erases;
- the skill of choosing *what* to free-parameterise is the skill of legal analysis itself.

So: we will not "solve" non-programmer programming by inventing one more surface syntax. We can lower the rung of the first ladder and rename the rungs into vocabulary drafters already trust. That is the only defensible promise.

7.2 The Pedagogy we believe matches Matthew's review criteria

Level 0 — Currently available in spirit (Visualiser / propositional L4-lite).

Closed atomic claims connected by AND / OR / NOT / IF. No variables. This is already within reach of careful policy readers after a short demo.

Level 1 — The next baby step (types as defined terms; predicates as completable sentences). *This is the GCO encoding's native level.*

Build only from what drafters already master:

They already know	Bridge phrase	Formal object
Defined terms	"write the interpretation schedule as a <i>form</i> "	type / record
Repeated nouns used consistently	"every slot on the form must be filled when the rule applies"	fields
<i>a</i> then <i>the</i>	"open a form instance; then refer back to it"	binding / parameter
Persons vs premises vs officers	"different kinds of card; you cannot put a venue where a person goes"	distinct types
Exclusive classifications (indoor/outdoor)	"choose one of these labels"	sum types
Counts-as	"when these facts hold, the law <i>treats it as</i> a restricted gathering"	constitutive predicate
Act / omission	"someone <i>did</i> X / <i>failed to</i> Y"	fact predicates
Must / must not / may	"what the rule requires of a person <i>given</i> the form is filled"	deontic / duty-fact predicates
Validity / availability of a defence or exception	a separate job-application form, not a special magic bit	boolean residual + named gate

Teaching line we are willing to stand behind:

You are not inventing *variables*. You are writing the *interpretation schedule as the form the paper rule already implies*. A *gathering* opens a form. *The gathering* points back at that form. A predicate is a *completable sentence* — Level 0 boxes were *closed* sentences; Level 1 opens the slots.

Level 2 — Relations across multiple forms. Person servee × gathering. Still first-order, still contiguous with draftsmanship of multi-party provisions.

Level 3 — Process (deadlines, HENCE/LEST). Only when the source reaches for it. Bad default for GCO-class instruments.

What we refuse to wave in front of tech-phobic drafters at the first sitting: lambdas, polymorphism, recursive list manipulators, "maps and folds", or any claim that residual ministerial guidance will be auto-resolved by the language.

7.3 Institutional / product openings still on the table

- Guided *form builders* that emit Level-1 L4 (or VisExpr-plus-arguments) without showing source until the author opts in.
- Automatic "this test case flipped under the outdoor-20 amendment" reports when two version files share the same golden suite.
- Back-links from exported evaluation traces to the § that decided the outcome.
- Honest UI language that never says "type error" when it can say "you asked a Person question of a Premises card".

These are product and education workstream, not another keyword in the parser.

8. Answers in one breath (the "Waddington FAQ")

Could this Order be encoded?

Yes. It has been. The encoding type-checks, evaluates, and carries golden cases that document amendment deltas.

Does that prove L4 is necessary?

No. Any careful formal language could encode the *decision procedure*. L4 (or a peer legal CNL) is what makes the encoding *reviewable as something that still looks like the Order*.

Does VS Code solve non-programmer authorship?

No. It solves *manufacture, checking, search, and evaluation*. Authorship remains a craft plus a pedagogy. Telling people otherwise is how RaC pilots lose trust.

Did you fully automate compliance decisions for Jersey at the material time?

No. Critical-worker and tour-guide residual inputs must be supplied. That is a feature: open texture and external guidance stay outside the machine until a competent human decides them.

Did you force a heavy deontic process model onto a light crime-and-exception order?

No. Art.3/7 are boolean duty-facts feeding offence predicates. The heavier regulative machinery was left on the shelf for instruments that need it.

Is "programming for non-programmers" solved?

No — and treating it as a language-syntax contest will keep failing. The defensible path is Level-0 propositional boxes → Level-1 forms and completable sentences named in legislative English → optional Level-3 processes. L4 already sits comfortably under Levels 1 and 3; Level 0 is the Visualiser / VizExpr projection. The remaining work is *bridges and vocabulary*, not more operators.

So what was won?

An auditable, versioned, residual-honest, test-backed model of a real Jersey COVID control order, published as callable decision functions, written so that a statute-natural reading path is available. That is maturer

than a slide that says "rules as code". It is still unfinished relative to the utopian ask of "drafters author alone on day one".

9. How to read the files

1. Open `GC0-as-repealed.l4` for the last-in-force model.
 2. Skim `§§ 1 Interpretation` first — that is the ontology.
 3. Walk `Art.3` → exceptions 4/5/6 → restricted / must-not → `Art.7` → `Art.9` offences in statutes order; the file follows that walk.
 4. Diff against `GC0-first-version.l4` with attention to `-- DIFF:` markers and the outdoor-of-11 golden case that flips outcome.
 5. Run the assertions / evals in-editor (`l4 run` or the VS Code L4 extension).
 6. Call the `@export` functions via the deployment / MCP surface if you want the "what would a front-line form ask?" experience.
-

10. Credits and posture

This encoding is a response to a challenge Matthew posed for instruments of this kind: do not hand-wave; encode something real; be frank about residual; do not pretend the IDE eliminated legal judgement; show version change as version change; keep the surface reviewable.

We agree with the thrust of that challenge. Where the encoding succeeds, it succeeds by treating the Order as already almost formal. Where it fails to "make programming easy", it fails for the reason every careful formaliser already knows:

Making precise things easy for people who never wished to become manufacturers of precision is not a weekend feature request. It is a long educational and product problem. Tooling can shorten the climb. It cannot repeal the mountain.

L4 encoding and report prepared in the Legalese / L4 workspace. Not legal advice; encoding for analytical illustration of a repealed subordinate instrument.